Test-ExecutableEnvironment

Local safety check for this PC, with actions for you and IT

Source: Test-ExecutableEnvironment.md · Generated 2026-11-12

How to run the script

1. Save `Test-ExecutableEnvironment.ps1` to a folder on your PC, e.g. `Documents\Checks`.
2. Open the Start menu, type **PowerShell** and open **Windows PowerShell**.
3. Type the command below, with the path to the downloaded file you want to check, and press Enter:

```
powershell -ExecutionPolicy Bypass -File "$env:USERPROFILE\Documents\Checks\Test-
ExecutableEnvironment.ps1" -Path "$env:USERPROFILE\Downloads\setup.exe"
```

The script only reads the file; it never runs it. The sections below explain every result and what to do about it.

Checks a downloaded Windows executable **against this PC** before you run it: where it came from, whether antivirus finds anything, whether App Control would block it, and whether its location is safe. For every problem it says **what you, or your IT administrator, can do**. The file is never executed.

Why a local check

Everything this script looks at depends on **your PC and how you got the file**:

- how it was downloaded (Mark of the Web, hash),
- the state of your antivirus,
- the App Control, Smart App Control and AppLocker policies on this machine,
- the folder and permissions the file sits in.

You can deal with these problems yourself, or with your IT department: download again, compare the hash, update or run antivirus, move the file to a safe folder, ask IT for an App Control rule. None of them need the publisher to change the file.

Contents

- Why a local check
- Quick start
- Parameters
- Output, actions and exit codes
- Findings and actions
- How each check works
  - 1. File identity
  - 2. Microsoft Defender scan
  - 3. App Control (WDAC, Smart App Control, AppLocker)
  - 4. File permissions
- Mark of the Web explained

- Worked example
- Limitations

Quick start

```
powershell -ExecutionPolicy Bypass -File .\Test-ExecutableEnvironment.ps1 -Path
"$env:USERPROFILE\Downloads\setup.exe"
```

With the hash the vendor publishes:

```
.\Test-ExecutableEnvironment.ps1 .\setup.exe -ExpectedSha256 3A7B... -ReportPath .\local-report.csv
```

Works in Windows PowerShell 5.1 and PowerShell 7. It runs without Administrator rights. Running as Administrator unlocks two extra details: the Code Integrity event log and the full CiTool policy list.

Parameters

Parameter	Purpose
`-Path` (required)	The file to check.
`-ExpectedSha256`	The SHA-256 hash the vendor publishes. A mismatch is a FAIL.
`-SkipDefenderScan`	Skip the Microsoft Defender scan (faster).
`-ReportPath`	Save the findings and actions to a `.json` or `.csv` file.

Output, actions and exit codes

Each finding has a level, and every WARN or FAIL is followed by one or more **actions**, each addressed to whoever can carry it out:

```
  [WARN] Signature has no timestamp. It stops being valid when the certificate expires on 2027-08-17
(278 days).
        -> Publisher: Add a timestamp now without re-signing: jsign timestamp --tsaurl <TSA> --tsmode
RFC3161 file.exe ...
        -> You: It works today. After 2027-08-17, Smart App Control blocks it and signature checks
fail; ...
```

Level	Meaning
`PASS`	The check succeeded.
`INFO`	A fact worth knowing. It may carry an optional action.
`WARN`	Something to fix or review.
`FAIL`	A serious problem. Don't run the file until it's fixed.

At the end, the **Summary** lists the actions again, grouped by owner, failures first, without duplicates. That makes it a ready-made to-do list per person. With `-ReportPath`, the JSON report holds each finding with its `Actions` (owner and text), and the CSV report has an `Actions` column (`Owner: text | Owner: text`).

Exit code	Verdict	Condition
`0`	No problems	Only PASS and INFO
`1`	Review warnings	At least one WARN, no FAIL
`2`	Not acceptable / do not run	At least one FAIL

Actions here are addressed to **You** (things you can do on this PC), **IT admin** (things that need an administrator or a policy change), and occasionally **Publisher**, where the only fix is to ask them.

Findings and actions

Every WARN and FAIL the script can report, and what to do:

Finding	Level	Who	Action
SHA-256 doesn't match `-ExpectedSha256`	FAIL	You, then Publisher	Delete it and download again over HTTPS; check you copied the hash for the right version and architecture. If a fresh copy still doesn't match, tell the publisher.
Hidden Unicode direction characters in the name	FAIL	You	Don't run it; delete it.
Double extension (`report.pdf.exe`)	FAIL	You	Don't run it; delete it. Turn on *File name extensions* in Explorer.
Hidden/System attributes	WARN	You	Find out why before running it (`attrib -h -s <file>` shows it).
Program with a non-program extension	WARN	You	Don't rename it to `.exe` to run it; ask the sender why.
Not a Windows PE file	WARN	You	Check what kind of file it really is.
Restricted-sites zone	WARN	You	Get the file from the publisher's official site instead.
No Mark of the Web	WARN	You	If it came from the internet, download it again with a browser or extract it with Windows' built-in zip support. Otherwise rely on the antivirus scan and the hash.
Extra hidden data stream	WARN	You	Inspect it with `Get-Content -Stream <name>`; remove it if unexpected with `Remove-Item -Stream <name>`.
Defender real-time protection off	WARN	You, IT admin	Turn it on in Windows Security, or confirm another antivirus or EDR is active.
Defender definitions more than 7 days old	WARN	You	`Update-MpSignature`, then run the check again.
Defender detected a threat	FAIL	You	Don't run it. Delete it (or let Defender quarantine it) and run `Start-MpScan -ScanType FullScan`.
Defender scan didn't complete / `MpCmdRun.exe` missing	WARN	You	Scan it manually (right-click > Scan) or with your installed antivirus.
WDAC enforced, file not validly signed	WARN	IT admin, Publisher	IT: allow it by hash in a supplemental policy if approved. Ask the publisher for a correctly signed build.
WDAC enforced, file validly signed	WARN	IT admin, You	IT: confirm a publisher rule allows the signer. You: if it's blocked, send IT event 3077 from the CodeIntegrity log.

Finding	Level	Who	Action
Smart App Control on, file not validly signed	WARN	You, Publisher	Don't turn Smart App Control off; it can't be turned back on without resetting Windows. Ask the publisher for a validly signed build.
Earlier Code Integrity block or audit events	WARN	IT admin	Review the event (it names the policy and rule) and add a rule if the file is approved.
AppLocker would deny it	WARN	IT admin	Add an AppLocker publisher or hash rule if the file is approved.
Execute explicitly denied	WARN	IT admin	A deny rule is usually deliberate; ask IT.
No Execute permission	WARN	You	Move it to a folder you own, or `icacls <file> /grant <you>:(RX)`.
Everyone/Users can modify the file	WARN	You	Move it to a new folder in your profile and run the check again there.
Folder writable by Everyone/Users	WARN	You	Move the file to a new folder in your profile.
DLLs next to the program	WARN	You	Move the program to its own empty folder before running it.

INFO findings with actions: no expected hash given (re-run with `-ExpectedSha256`), Mark of the Web present (check `HostUrl` is the publisher's real domain; don't `Unblock-File` yet), Defender in passive mode (also scan with your main antivirus), missing ASLR/DEP (consider Windows Sandbox), Code Integrity log unreadable (run as Administrator).

How each check works

1. File identity

Hash. `Get-FileHash -Algorithm SHA256` gives the file's fingerprint. If you pass `-ExpectedSha256`, the two are compared. A match proves the file is byte-for-byte what the vendor published. The script also prints a VirusTotal lookup link built from the hash. It doesn't upload anything; you open the link yourself.

Deceptive names. Two common tricks are flagged as FAIL:

- Hidden Unicode direction characters (e.g. U+202E, "right-to-left override") that make `invoice[U+202E]fdp.exe` display as `invoiceexe.pdf`.
- Double extensions such as `report.pdf.exe`. Explorer hides known extensions by default, so this shows as `report.pdf`.

Hidden or System file attributes are a WARN.

PE header. The script reads the first bytes of the file directly with a `BinaryReader`, without loading or running it:

- `MZ` at offset 0 and `PE\0\0` at the offset stored at `0x3C` confirm it's a Windows executable.
- The *Machine* field gives the architecture (x86, x64, ARM64).
- The *Characteristics* field says whether it's a DLL.
- The *Subsystem* field says whether it's a GUI or console program.
- A non-empty CLR data directory (entry 14) means it's a .NET assembly.
- *DllCharacteristics* shows whether ASLR (`0x40`) and DEP/NX (`0x100`) are enabled. Missing ones are reported as INFO; modern compilers set both.

A PE file with an unexpected extension (e.g. a `.jpg` that is really an `.exe`) is a WARN.

Version info. `[Diagnostics.FileVersionInfo]::GetVersionInfo()` reads the CompanyName, ProductName, FileDescription, FileVersion and OriginalFilename embedded by the developer. These are self-declared, so anyone can write anything there. `Test-ExecutableSignature.ps1` cross-checks the CompanyName against the signer.

Mark of the Web. Reads the `Zone.Identifier` stream to learn the download zone and source URL. See Mark of the Web explained. A missing mark is a WARN, because it means SmartScreen and Smart App Control won't check the file.

Extra data streams. `Get-Item -Stream *` lists all NTFS alternate data streams. Anything other than the main data, `Zone.Identifier` and `SmartScreen` is a WARN, because hidden streams can carry a payload.

2. Microsoft Defender scan

- `Get-MpComputerStatus` reports whether Defender is the active antivirus or in passive mode, whether real-time protection is on, and how old the definitions are. Definitions more than 7 days old are a WARN.
- The script finds the newest `MpCmdRun.exe`, under `C:\ProgramData\Microsoft\Windows Defender\Platform\<version>\` with a fallback to `Program Files`, and runs:

  ```
  MpCmdRun.exe -Scan -ScanType 3 -File <path> -DisableRemediation
  ```

  `-ScanType 3` scans just the one file. `-DisableRemediation` reports any threat without quarantining or deleting the file.
- Exit code `0` means no threats (PASS) and `2` means a threat was found (FAIL). Anything else means the scan didn't complete (WARN).

3. App Control (WDAC, Smart App Control, AppLocker)

Whether a policy blocks a file depends partly on its signature. This script **reads** the signature status (Valid, NotSigned, ...) only as an input to the prediction and reports it as INFO. It doesn't judge the signature or the certificate; `Test-ExecutableSignature.ps1` does that.

This section predicts whether Windows will **block** the file. Windows can't test a file against an App Control policy without running it, so this is a prediction from the policy state and the signature, not a guarantee.

App Control for Business (WDAC). Queries the `Win32_DeviceGuard` WMI class in `root\Microsoft\Windows\DeviceGuard`:

- `CodeIntegrityPolicyEnforcementStatus` covers kernel mode (drivers).
- `UsermodeCodeIntegrityPolicyEnforcementStatus` covers user mode (UMCI). **This one decides whether `.exe` files may run.**

Values: `0` Off, `1` Audit, `2` Enforced. The prediction:

UMCI	File signature	Result
Off	any	PASS: WDAC won't block it.
Audit	any	INFO: won't block, but logs event 3076 if it would have.
Enforced	not valid	WARN: block expected unless a policy allows its hash.
Enforced	valid	WARN: runs only if a policy trusts this signer, its hash or a managed installer. Ask your administrator.

On Windows 11 22H2 and later, `CiTool.exe --list-policies --json` lists the active Code Integrity policies by name. Windows' own built-in policies, such as the vulnerable driver blocklist, appear here too and only affect drivers.

Smart App Control (Windows 11 consumer feature). Read from the registry value `HKLM\SYSTEM\CurrentControlSet\Control\CI\Policy\VerifiedAndReputablePolicyState`: `0` Off, `1` On, `2` Evaluation. When it's on, unsigned or invalidly signed files are likely blocked unless Microsoft's cloud already knows them as safe. Validly signed files from a trusted CA are generally allowed.

Past block events. Searches the `Microsoft-Windows-CodeIntegrity/Operational` event log for the last 30 days for events that mention the file name:

Event ID	Meaning
3077	Blocked by an enforced App Control policy
3076	Would have been blocked (audit mode)
3033	Blocked: didn't meet the required signing level
3034	Would have been blocked: signing level (audit mode)

Reading this log needs Administrator rights.

AppLocker. Exports the effective policy with `Get-AppLockerPolicy -Effective -Xml` and checks whether any Exe, Msi or Dll rules exist. If they do, the policy is saved to a temporary XML file and the script runs:

```
Test-AppLockerPolicy -XmlPolicy <temp.xml> -Path <file> -User <you>
```

This is a real dry run: it returns `Allowed`, `Denied` or `DeniedByDefault` without executing anything. The script also reports each collection's enforcement mode and whether the Application Identity service (`AppIDSvc`) is running; AppLocker only enforces when that service runs. The AppLocker module doesn't exist on Windows Home, so there it's reported as not available.

4. File permissions

The ACL is read with `Get-Acl` and `GetAccessRules(..., [SecurityIdentifier])`. Using SIDs instead of names makes the checks work in any Windows language.

- **Owner and every access rule** are listed as INFO.
- **Can you execute it?** Your token's SIDs (your user SID plus every group from `WindowsIdentity.GetCurrent().Groups`) are matched against the rules. Allow and Deny rights are combined. A Deny on `ExecuteFile` (`0x20`) is a WARN because the file won't start. No Allow for it is also a WARN. This is an approximation; Windows' own access check also considers integrity levels and privileges.
- **Can others replace it?** If a broad group has write, delete, change-permission or take-ownership rights on the file, that's a WARN. Another user or process could swap the file between your check and your run (a "time-of-check to time-of-use" attack). The broad groups are Everyone (`S-1-1-0`), Authenticated Users (`S-1-5-11`), Users (`S-1-5-32-545`), INTERACTIVE, Guests, ANONYMOUS LOGON and NETWORK.
- **Can others plant DLLs?** The same check runs on the containing folder (create files or folders rights). Windows searches a program's own folder first when loading DLLs, so a writable folder lets someone plant a malicious DLL ("DLL hijacking").
- **Stray DLLs next to it.** If the program sits next to `.dll` files in a user folder such as Downloads, that's a WARN, because an old malicious download could be side-loaded. Move the program to its own empty folder before running it. System folders (`Windows`, `Program Files`) are skipped.

Mark of the Web explained

Mark of the Web (MotW) is the tag Windows attaches to files that come from the internet. It isn't stored in the file's contents. It's in a hidden **NTFS alternate data stream** named `Zone.Identifier`. An NTFS file can have several named streams; the normal contents live in the unnamed `:$DATA` stream, and the mark sits beside it without changing the file's size or hash.

Example contents:

```
[ZoneTransfer]
ZoneId=3
ReferrerUrl=https://example.com/downloads/
HostUrl=https://cdn.example.com/files/setup.exe
```

Field	Meaning
`ZoneId`	Security zone the file came from: `0` this computer, `1` intranet, `2` trusted sites, `3` internet, `4` restricted sites
`HostUrl`	The exact URL the file was downloaded from
`ReferrerUrl`	The page you were on when the download started

Who writes it. The program that saves the file: browsers (Edge, Chrome, Firefox), Outlook, Teams and Windows' built-in zip extraction, using the Attachment Execution Service (`IAttachmentExecute`) or by writing the stream directly. `curl`, copies from USB sticks and some archive tools don't add it.

Who reads it. SmartScreen ("Windows protected your PC"), Smart App Control, Office Protected View and macro blocking, PowerShell's `RemoteSigned` execution policy, and Explorer's *Unblock* checkbox.

Not passed on to extracted or installed files. Files a program writes, such as an installer extracting its payload, don't get the mark. See Embedded executables have no Mark of the Web (in *Test-ExecutableSignature.pdf*).

How it's lost.

- `Unblock-File` or Properties → Unblock deletes it.
- Copying to FAT32 or exFAT drives or some network shares drops it, because those have no alternate streams.
- Some archive tools don't pass it on to files they extract. Attackers use this to bypass SmartScreen.

See it yourself:

```
Get-Item .\file.exe -Stream *                    # list all streams
Get-Content .\file.exe -Stream Zone.Identifier   # read the mark
```

In `cmd.exe`, `dir /r` shows the streams.

The mark is a clue, not proof. Anyone who can write to the file can edit or delete it.

Worked example

An unsigned test program downloaded from the internet. A hidden data stream was added to it, a stray DLL was left next to it, and the wrong hash was given on purpose. The output is shortened:

```
== File ==
  [FAIL] SHA-256 does NOT match the expected hash (DEADBEEF).
         -> You: Delete the file and download it again from the official site over HTTPS. Check you
copied the right hash (version, architecture).
         -> Publisher: If a fresh download still does not match, tell the publisher: their download or
published hash is wrong.
  [INFO] Mark of the Web present: Internet. SmartScreen and Smart App Control will check it at launch.
  [INFO] HostUrl: https://example.com/agent.exe
  [WARN] Extra hidden data stream 'hidden' (3 bytes).
         -> You: Inspect it: Get-Content -LiteralPath '...\agent.exe' -Stream 'hidden'. Remove it if
unexpected: ...

== App Control for Business (WDAC), Smart App Control, AppLocker ==
  [INFO] Signature status used for the prediction: NotSigned. Details: Test-ExecutableSignature.ps1
  [PASS] No enforced user-mode App Control policy: WDAC will not block this file.

== File permissions ==
  [WARN] 1 DLL file(s) sit next to the program (stray.dll). A program can load DLLs from its own folder.
         -> You: Create a new folder in your own profile (e.g. %USERPROFILE%\Installers\<name>), move
the file there, and run it from there. ...

== Summary ==
  Actions for YOU (on this PC):
    [FAIL] Delete the file and download it again from the official site over HTTPS. ...
    [WARN] Inspect it: Get-Content ...
    [WARN] Create a new folder in your own profile ...
  Ask the PUBLISHER:
    [FAIL] If a fresh download still does not match, tell the publisher: their download or published
hash is wrong.

  VERDICT: DO NOT RUN. One or more local checks failed (see actions above).
```

Everything here can be fixed locally: download again, remove or inspect the stream, move the file. The file being unsigned is only used for the App Control prediction; whether that's acceptable is for `Test-ExecutableSignature.ps1`.

Limitations

- **No check can prove a file is safe.** A clean result means none of these checks found a problem.
- **The signature isn't judged here.** A file can pass every local check and still have a broken, untrusted or untimestamped signature. Run `Test-ExecutableSignature.ps1` too.
- **App Control results are predictions.** Windows has no dry-run API for WDAC or Smart App Control; only AppLocker can be truly tested.
- **SmartScreen reputation can't be queried** from a script; it's checked when the file is launched.
- **The execute-permission check is an approximation** based on the ACL and your groups.
- Some details (the Code Integrity log, the full CiTool output) need Administrator rights.